

AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ FIZYKI I INFORMATYKI STOSOWANEJ

KATEDRA INFORMATYKI STOSOWANEJ I FIZYKI KOMPUTEROWEJ

Projekt dyplomowy

Self-sufficient library for calculating ground state energy of molecules on quantum computers

Samowystarczalna biblioteka do obliczania energii stanu podstawowego molekuł na komputerach kwantowych

Autor: Martyna Migdałek
Kierunek studiów: Fizyka Techniczna
Opiekun pracy: dr inż. Paweł Szumniak

Kraków, 2024

I would like to express my deepest gratitude Dr. Tomasz Stopa (IBM Rapid Network Automation — IBM Quantum Ambassador) from IBM Software Lab, Krakow, Poland, for his valuable support during the realization of my engineering thesis. His insightful discussions, ideas, and continuous guidance have been essential throughout this project.

Contents

Introduction **4**

1 Theoretical background **5**
1.1 From roots to present 5
1.1.1 Complexity 5
1.2 Computational quantum chemistry 6
1.2.1 Electronic Structure Problem 6
1.2.2 Hartree-Fock approximation 8
1.3 Second quantization notation and mapping 9
1.4 VQE algorithm 11

2 Software environment for quantum research **13**
2.1 Qiskit 13
2.2 Qiskit Nature 13
2.3 PySCF 14

3 Computing ground-state energies of molecules **15**
3.1 Hydrogen 15
3.1.1 Circuits 16
3.1.2 Optimizers 18
3.1.3 Results 19
3.1.4 Simulators 22
3.2 Analysis in relation to other molecules 23

4 Replacement for drivers **27**

Summary **30**

Bibliography **30**

Introduction

Determining an appropriate problem for a quantum computer to address is a sophisticated and challenging task. Uncomplicated dilemmas can be resolved with classical computing. Nevertheless, dealing with issues that require a significant count of qubits alongside extensive circuit depth or demand error corrections is not achievable by present-day quantum technology. The objective is to identify the issue situated between these two extremes, using the maximum number of available qubits, incorporating error mitigation, and potentially achieving quantum advantage. Emerging areas with good prospects include the management of complex structured data, optimization strategies, and nature simulation. The most promising results among the latter field are material simulation and chemical modeling, which hold potential for development in the fields of carbon dioxide capture, pharmaceuticals, or even the invention of new types of batteries.

The simulation of chemical systems serves as a fundamental focus for quantum computing. According to Matthias Troyer, representing all potential configurations of 125 orbitals would necessitate more memory than the total number of atoms in the entire universe. On quantum hardware, it would require 250 qubits, which exceeds the current availability. However, this should not deter us from refining and developing algorithms that can be optimally employed when this capability is achieved.

The goal of this project is to gain basic knowledge about quantum software, the semiquantum algorithm VQE, and to explore the field of computational chemistry, which represents one of the most impactful and rapidly advancing areas in the development of quantum technologies. I would like to perform benchmark calculations on the ground state energy of small molecules (H_2, LiH, Li_2) to validate the efficiency and precision of Qiskit Nature. Additionally, investigate how the choice of optimizers and mappers influence the results and quantum circuit. The final step will involve the development of a new package containing tabulated data independent of external libraries like PySCF or Gaussian. This approach in its initial stage would involve generating and storing key computational results in advance. The newly developed package has been tested and can be further integrated with the existing Qiskit library.

Chapter 1

Theoretical background

1.1 From roots to present

Over 40 years ago Feynmann presented his reflection on the imitating physics with computers [1]. In his opinion, the primary challenge resides in the simulation of quantum systems. He had first tried doing it on a classical computer and came to the conclusion that it could not keep up with an increase of computational cost related to exponential growth in the complexity of the quantum model. That is how the idea of quantum hardware was created. Device that does not try to copy laws of quantum physics but instead operates in alignment with them.

Even before building the first digital computer, Turing [2] stated about the possibility of creating *the universal computing machine* that could compute anything with an accurate amount of resources. There is a quantum equivalent of it: *the universal quantum computer* presented by David Deutsch [3]. Its task is to replicate every physical system perfectly. There is a possibility that we will be able to conduct more advanced and realistic simulations, further validating experimental outcomes or even offering additional detailed insights.

Currently, smaller quantum systems are being used for some purposes. We could notice the same examples in the analysis of multiphoton entangled states [4] and simulations involving ion-trap quantum gates [5] or materials and electronic structure [6].

Assertions are frequently made that quantum computing possesses the potential to provide meaningful speed enhancements over classical methods for certain problem domains. We are now stepping into an era in which quantum computers are beginning to work comparable to classical computers (quantum utility). IBM's scientists stated that prior to the construction of fault-tolerant quantum circuits, there exist proofs that show the value of quantum computations within physical models [7]. Quantum processors exceeding 100 qubits, such as the Eagle with 127 qubits and Heron with 133 qubits, which cannot be classically emulated, are available for use.

1.1.1 Complexity

A significant challenge encountered by researchers involves the selection of a suitable algorithm to effectively address the problem. The difficulty of an issue is defined

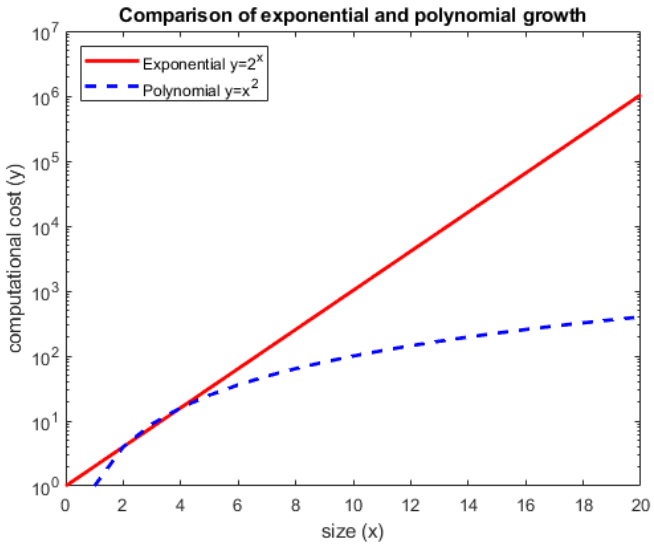


Figure 1.1: Graph showing the increase in computational cost for problems NP and P on a logarithmic scale

by the feasibility of resolving it in polynomial time. If we are provided with a solution to an NP (non-deterministic polynomial time) problem, its correctness is quickly ascertained. The complexity of solving some of these problems intensifies exponentially with the number of input variables. In the image 1.1, we can observe a comparative analysis of these two types of growth. Although they align for a small amount of data, the increase in NP problems significantly outpaces that in P problems, demonstrating a much steeper trajectory. More complexity classes have been presented for problems in quantum chemistry and physics [8]. The BQP class consists of problems that can be easily computed on quantum hardware. The QMA given a proposed solution encoded as a quantum state is able to effectively validate the solution, and QMA-complete even with the capabilities of a quantum computer is improbable to find an efficient solution.

1.2 Computational quantum chemistry

The employment of computers to evaluate a simplified representation of a chemical system is referred to as computational chemistry. Provided a suitable theoretical background, we can simulate properties like energies, structures (equilibrium bond lengths, proteins) or spectra (IR, NMR, UV Vis etc.).

1.2.1 Electronic Structure Problem

This field is primarily pertains to describing the electronic structure, which is an arrangement of electrons in molecules with the help of quantum mechanics ([9], [10], [8]). We are able to characterize the majority of chemical system by time-independent, non-relativistic Schrodinger equation:

$$\hat{H} \left|\Psi\right\rangle = E \left|\Psi\right\rangle . \tag{1.1}$$

Table 1.1: Representative examples of problems classified as BQP, NP-complete, and QMA-complete based on [8]

BQP Problems	**NP-complete Problems**	**QMA-complete Problems**
Unitary time evolution simulation	Determining the ground state in 3D Ising models	The N-representability problem
Estimating Kohn-Sham potential in time-dependent DFT	Hartree-Fock calculations for general quantum systems	Determining the universal functional in DFT
Quantum phase estimation techniques	Hartree-Fock on systems with translational symmetry	Problems involving the Bose-Hubbard model

that encompasses the molecular Hamiltonian, molecular wave function and total molecular energy. Full system's molecular energy is determined by the Coulomb Hamiltonian that is a sum of kinetic energies $\hat{T}$ from nuclei and electrons, potential energies $\hat{U}$ from attractive or repulsion force between electrons and nucleus:

$$\hat{H} = \hat{T}_n + \hat{T}_e + \hat{U}_{en} + \hat{U}_{ee} + \hat{U}_{nn}, \tag{1.2}$$

$$\begin{aligned} \hat{H} = -\sum_I \frac{1}{2M_I}\nabla^2_{R_I} - \sum_i \frac{1}{2}\nabla^2_{r_i} - \sum_i \sum_J \frac{Z_J}{|r_i - R_J|} + \\ + \sum_i \sum_j \frac{1}{|r_i - r_j|} + \sum_I \sum_J \frac{Z_I Z_J}{|R_i - R_j|}, \end{aligned} \tag{1.3}$$

where the Hamiltonian is written in atomic units (mass of the electron $m_e = 1$, charge of electron $e = 1$, the reduced Planck constant $\hbar = 1$, electric constant $4\pi\epsilon_0 = 1$) and symbols indicate M mass, R position, Z atomic number of the nucleus and r position of the electron. Considering this equation is many-body problem the complexity of it scales exponentially, it is challenging or even impracticable to get exact solution. Consequently, it is frequent to use the Born-Oppenheimer approximation. It assumes that since electrons are magnitudes lighter than nuclei, they also move relatively faster than the nuclei, which we can fix in space. That means kinetic energy of nuclei $\hat{T}_n$ is equal to zero, repulsion nucleus energy U_{nn} is constant and remain invariant and unaltered with respect wavefunction. Taking that into consideration we reduce our problem to the resolution of the electronic Schrödinger equation:

$$\hat{H}_{el} \left|\Psi_{el}\right\rangle = E_{el} \left|\Psi_{el}\right\rangle, \tag{1.4}$$

$$\hat{H}_{el} = \hat{T}_e + \hat{U}_{en} + \hat{U}_{ee}. \tag{1.5}$$

By selecting an alternative geometric configuration of a given molecule, we may observe modifications in both its energy and electronic structure

1.2.2 Hartree-Fock approximation

Before we proceed to the formulation of the n-electron wave function, it is helpful to clarify some theoretical aspects.

a) An orbital - one-electron wave function characterizes the location of an electron (occupied with an electron, virtual without an electron). When an orbital specifies coordinates, it is called spatial orbital, if it includes a representation of spin, it is termed a spin orbital:

$$\chi(\vec{x}_1) = \phi(\vec{r}_1)\sigma(\omega_1), \tag{1.6}$$

where $\vec{x}_1 = (\vec{r}_1, \vec{\omega}_1)$ is a vector of particular electron's properties, $\phi(\vec{r}_1)$ is a spatial orbital and $\sigma(\omega_1)$ is a spin function (spin up or down).

b) Anti-symmetry of electronic wavefunctions - to exchange positions of two electrons in wave function, it is necessary to put a minus sign in front of it:

$$\Psi(\vec{x}_1, ..., \vec{x}_a, \vec{x}_b, ..., \vec{x}_n) = -\Psi(\vec{x}_1, ..., \vec{x}_b, \vec{x}_a, ..., \vec{x}_n). \tag{1.7}$$

To express two-electron wavefunction in alignment with this principle, we might use linear combination:

$$\Psi(1,2) = \frac{1}{\sqrt{2}}(\Psi_1(1)\Psi_2(2) - \Psi_2(1)\Psi_1(2)), \tag{1.8}$$

$$\Psi(2,1) = \frac{1}{\sqrt{2}}(\Psi_1(2)\Psi_2(1) - \Psi_2(2)\Psi_1(1)) = -\Psi(1,2). \tag{1.9}$$

The normalization constant is equal to $\frac{1}{\sqrt{n!}}$. As the number of electrons n increases, there is a corresponding expansion in the number of terms in a factorial manner.

c) Slater determinants represent a more systematic approach to formulate the previously discussed wavefunction:

$$\Psi_{el} = \frac{1}{\sqrt{n!}} \begin{vmatrix} \chi_1(1) & \cdots & \chi_n(1) \\ \vdots & \ddots & \vdots \\ \chi_1(n) & \cdots & \chi_n(n) \end{vmatrix}. \tag{1.10}$$

As the column changes, the spin orbitals are altered, whereas changes in the row correspond to subsequent coordinates of individual electrons. This form of describing a wave function adheres to the Pauli exclusion principle, implying that if any two electrons are in the same orbital, the whole expression becomes zero.

Assigning the following electrons to the lowest-energy available spin orbitals will result in the lowest-energy Slater determinant, corresponding to the ground state:

$$|\Psi_0\rangle = |\chi_1, \chi_2, ..., \chi_n\rangle . \tag{1.11}$$

Any other combination of electrons is called an excited Slater determinant. When we move one electron to a higher state, it is singly excited, two electrons are doubly excited, etc.

Any n-electron wavefunction can be described as a linear combination of all Slater determinants:

$$|\Psi\rangle = C_0 |\Psi_0\rangle + \sum_{i=1}^{n} \sum_{j+1}^{2k} C_{ij} |\Psi_{ij}\rangle + ... \tag{1.12}$$

Equation 1.12 shows the initial two components: the ground state and single excited states. C_0 and C_{ij} are their coefficients. In computational chemistry spatial orbitals are often represented by finite set k basis functions, $2k$ symbolizes the number of all spin orbitals ($2k - n$ - virtual orbitals, n - occupied). The method that considers only the ground state is known as the Hartree-Fock approximation. This means that our wave function is represented by a single Slater determinant:

$$E_{el} = \langle \Psi_{el}| \hat{H}_{el} |\Psi_{el}\rangle \approx E_{el}^{HF} = \langle \Psi_0| \hat{H}_{el} |\Psi_0\rangle . \tag{1.13}$$

1.3 Second quantization notation and mapping

To formulate the electronic structure problem in a way that is comprehensible to a quantum computer, it must be transformed in some way. One of the most commonly used and relatively straightforward approaches is second quantization ([11]). The Hamiltonian 1.5 consists of operators that depend on one electron $\hat{T}_e$ and $\hat{U}_{en}$, and two electrons U_{ee} at a time. In order to articulate them in the second quantization, it is essential to first establish the creation $\hat{a}_j^\dagger$ and annihilation $\hat{a}_j$ operators for fermions. They enable electron movement between orbitals: $\hat{a}_j^\dagger$ adds an electron to certain orbital and $\hat{a}_j$ erases it. Electrons are subject to the Pauli exclusion principle, so these operators satisfy anti-commutation relations:

$$\{\hat{a}_i^\dagger, \hat{a}_j^\dagger\} = \hat{a}_i^\dagger \hat{a}_j^\dagger + \hat{a}_j^\dagger \hat{a}_i^\dagger = 0, \tag{1.14}$$

$$\{\hat{a}_i, \hat{a}_j\} = \hat{a}_i \hat{a}_j + \hat{a}_j \hat{a}_i = 0, \tag{1.15}$$

$$\{\hat{a}_i, \hat{a}_j^\dagger\} = \hat{a}_i \hat{a}_j^\dagger + \hat{a}_j^\dagger \hat{a}_i = \delta_{ij}. \tag{1.16}$$

General forms of one-body and two-body operators ([12]) in second quantization are:

$$\hat{O}_1 = \sum_{n=1}^{N} o_n = \sum_{i,j} \langle \mu_i| \hat{o} |\mu_j\rangle_n \, a_i^\dagger a_j, \tag{1.17}$$

$$\hat{O}_2 = \frac{1}{2} \sum_{n,m,n\neq m}^{N} \hat{o_{nm}} = \frac{1}{2} \sum_{i,j,k.l} \langle \mu_i| \langle \mu_j| \hat{o_{nm}} |\mu_k\rangle_n |\mu_l\rangle_m \, a_i^\dagger a_j^\dagger a_k a_l. \tag{1.18}$$

We can apply them to express our problem:

$$\begin{aligned} E_{el}^{HF} = &\sum_{i,j}^{N} \langle\phi_i| \hat{T}_e + \hat{U}_{en} |\phi_j\rangle \langle\Psi_{el}| \hat{a}_i^\dagger \hat{a}_j |\Psi_{el}\rangle \\ &+ \frac{1}{2} \sum_{i\neq j}^{N} \langle\phi_i| \langle\phi_j| \frac{1}{r_{12}} |\phi_k\rangle_{r_2} |\phi_l\rangle_{r_1} \langle\Psi_{el}| \hat{a}_i^\dagger a_j^\dagger \hat{a}_k \hat{a}_l |\Psi_{el}\rangle . \end{aligned} \tag{1.19}$$

For the sake of simplicity, we express this equation as:

$$\hat{H}_{el} = \sum_{ij} h_{ij} \hat{a}_i^\dagger \hat{a}_j + \frac{1}{2} \sum_{ijkl} h_{ijkl} \hat{a}_i^\dagger a_j^\dagger \hat{a}_k \hat{a}_l, \tag{1.20}$$

where h_{ij} and h_{ijkl} denote one and two body electronic integrals.

An additional issue that we can encounter is that our fermion-based system needs to be converted into qubit-based. This transformation enables the representation of the electronic structure problem to be a suitable form for quantum computation. There are several mapping techniques ([13], [14], [15]). Among these, the Jordan-Wigner mapping is the most straightforward approach to accomplish the desired transformation. In this mapping, fermionic operators are encoded as strings of Pauli operators. The mappings for creation and annihilation operators are given as:

$$\hat{a}_p^\dagger = \frac{1}{2} (X_p - iY_p) \prod_{j=0}^{p-1} Z_j, \tag{1.21}$$

$$\hat{a}_p = \frac{1}{2} (X_p + iY_p) \prod_{j=0}^{p-1} Z_j. \tag{1.22}$$

Parity mapping leverages the system's parity to streamline the implementation of operators, with the objective of minimizing computational costs:

$$\hat{a}_p^\dagger = \frac{1}{2} (X_p - iY_p) \prod_{j=p+1}^{n-1} Z_j. \tag{1.23}$$

$$\hat{a}_p = \frac{1}{2} (X_p + iY_p) \prod_{j=p+1}^{n-1} Z_j, \tag{1.24}$$

Each term in the Hamiltonian should be expanded according to the chosen mapping. After expansion, the Hamiltonian becomes:

$$\hat{H}_q = \sum_i c_i \hat{P}_i \tag{1.25}$$

where c_i is coefficient and $\hat{P}_i$ are the tensors products of Pauli operator[16].

1.4 VQE algorithm

Variational Quantum Eigensolver (VQE) [17] is a hybrid algorithm that combines two approaches classical and quantum. Its objective is to determine the lowest eigenvalue of the Hamiltonian. This approach is grounded in the Rayleigh-Ritz variational principle [8]:

$$\langle \Psi(\vec{\theta}) | \, H \, | \Psi(\vec{\theta}) \rangle \geq E_0, \tag{1.26}$$

where $|\Psi(\vec{\theta})\rangle$ is a trial wave function, E_0 represents the ground state energy eigenvalue of the Hamiltonian and $\vec{\theta}$ is a vector composed of independent parameters. The ground state wavefunction and energy are obtained by minimizing the energy expectation value by identifying parameters 1.2. The quantum computer's capabilities align well with rotating and entangling qubits (preparing the wavefunction). The expectation value of the Hamiltonian for this wavefunction is then measured. Parameters adjustment, which means updating angles for the unitary rotations, is accomplished through the application of optimization algorithms on a classical computer.

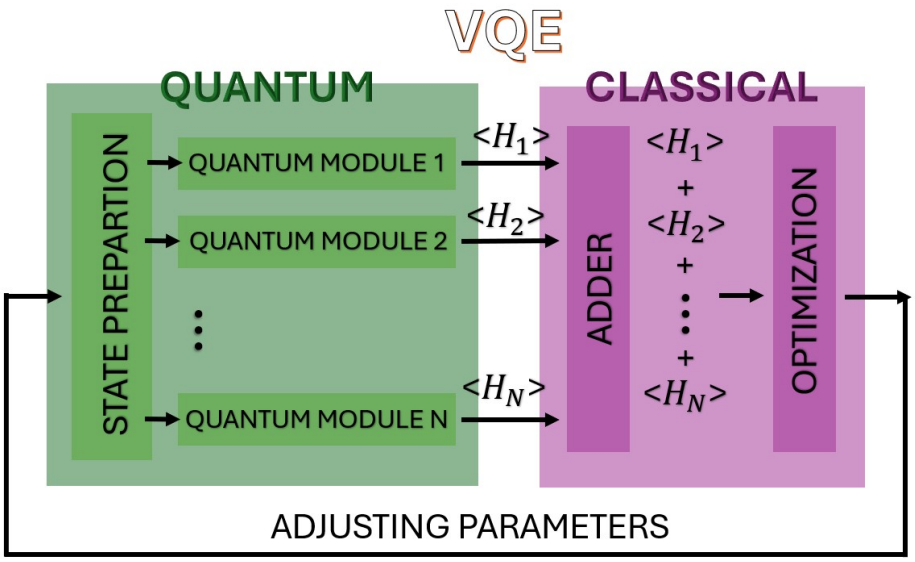


Figure 1.2: Scheme of VQE algorithm inspired by figure in [17]

One of the major obstacles is finding an adequate unitary transformation. It's general form ([18]) is:

$$\hat{U}(\tau) = \prod_{k=1}^{M} e^{i\tau_k \hat{P}_k}, \tag{1.27}$$

where τ_k is an amplitude optimized by VQE, M symbolize number of all $\hat{P}_k$ tensors smaller or equal to $4^N - 1$ (N - number of qubits). If we took into account every expression $\hat{P}_k$, the calculations would not be efficient. That is why there are methods that enable us to choose a part of them, such as the unitary coupled cluster ansatz ([19]). This fermionic picture theory is expressed by the equation:

$$\hat{U}_{CC} = exp(\sum_{n} \hat{T}_n - \hat{T}^{\dagger}), \tag{1.28}$$

in which T^n is a cluster operator containing n-excitation operator. In this work, the Trotter approximation will be employed, reducing this expression to the form of:

$$\hat{U}_{CCSD} = exp(\sum_{n=1}^{2} \hat{T}_n - \hat{T}^\dagger). \tag{1.29}$$

It is not possible to measure the expectation value of the whole Hamiltonian. It must be divided to parts([20], [21]). We are able to estimate only the single-qubit Pauli Z operator due to hardware limitation. To enable measurement of the other ones, it is necessary to apply a rotation to the system (example: Hadamard gate for the single-qubit X operator). This also works for their linear combination, but it is imperative that the operators commute. Otherwise, there is a possibility that this transformation may be excessively complex for a classical computer to manage, which is necessary to determine the appropriate operator to be estimated on the quantum hardware. To apply this rule to our qubit Hamiltonian, we must retain only the m parts that commute with each other:

$$\langle\Psi|\, \hat{H}_q \,|\Psi\rangle = \sum_m \langle\Psi|\, \hat{H}_m \,|\Psi\rangle\,. \tag{1.30}$$

Taking into account the definition of unitary operator:

$$\hat{U}^\dagger \hat{U} = I, \tag{1.31}$$

we proceed with the transformation:

$$\sum_m \langle\Psi|\, \hat{U}_m^\dagger \hat{U}_m \hat{H}_m \hat{U}_m^\dagger \hat{U}_m \,|\Psi\rangle = \sum_m \langle\Psi|\, \hat{U}_m^\dagger \hat{A}_m \hat{U}_m \,|\Psi\rangle = \sum_m \langle\Psi'_m|\, \hat{A}_m \,|\Psi'_m\rangle\,. \tag{1.32}$$

Chapter 2

Software environment for quantum research

2.1 Qiskit

Quantum Information Software Kit for short Qiskit [22] is an open-source software development kit specifically engineered for compatibility with quantum computers founded by IBM. Qiskit's runtime block is responsible for streamlining tasks and executing them effectively, and the primitives block is for designing a quantum workflow. The initial inquiry that arises is likely related to the working mechanisms of this platform [23]. The instructions from our computers are transmitted to the cloud, where they are decoded by specialized electronics maintained at ambient temperature. Microwave impulses (frequency around 1 GHz) produced by these devices travel along wires to the dilution refrigerator and subsequently to the quantum computer. IBM's systems are based on transmons [24], which are a type of superconducting qubits. It is worth mentioning that Qiskit provides access to computing with real quantum computers but also simulators powered by classical ones.

2.2 Qiskit Nature

Using the tools provided by Qiskit and its modules within the framework allows the resolution of some second quantization issues. Qiskit Nature [25] is responsible for tasks related to natural science. It contains chemistry packages to generate the molecular Hamiltonian and mappers that convert the second-quantized Hamiltonian into a qubit Hamiltonian. Furthermore, it seamlessly integrates with Qiskit's libaries for circuit definition, optimization and executing the VQE algorithm.

General construction contains problem (electronic or vibrational structure and lattice model problems), algorithm (ground-states, excited-states solvers, Hamiltonian simulation) and result.

```
result = algorithm.solve(problem)
```

Listing 2.1: General construction in Qiskit Nature

Calculating electronic integrals is necessary to obtain the fermionic operator. This can be accomplished with a program or library for computational chemistry. Qisikt Nature possess drivers that depend on three of them: Gaussian 16, Psi4 and

PySCF. In order to exploit any driver, you must install an adequate program. In this study, the results are calculated with the class `PySCFDriver`. In order to acquire the `problem` (`ElectronicStructureProblem`) after initializing the driver with the selected data set, it is necessary to run it.

```
from qiskit_nature.units import DistanceUnit
from qiskit_nature.second_q.drivers import PySCFDriver

geometry = 'Li .0 .0 .0; H .0 .0 1.595 ' #example molecule
driver = PySCFDriver( atom=geometry, unit=DistanceUnit.ANGSTROM, basis
    ='sto3g')
ESP = driver.run()
```

Listing 2.2: Example inicialization of the PySCFCDriver

In this example class `GroundStateEigensolver` that requires two arguments: mapper (some kind of qubit converter:`JordanWignerMapper`, `ParityMapper`) and solver (in this case `VQE` from qiskit_algorithms) is utilized for the purpose of receiving ground-state energy. The `solve` function that operates in this class takes an `ElectronicStructureProblem` as an instance, which is a data structure that describes a problem associated with the electronic Schrödinger equation (including the second quantized operators).

2.3 PySCF

The Python-based simulations of the chemistry framework library [26] [27] is a key component that allows us to compute electronic integrals. It is widely used in various fields, from electronic structure analysis to crystal modeling to developing quantum algorithms and computational methods for simulating chemical systems. The construction using Python facilitates the coding and debugging process. This library ensures high-performance computations through the integration of NumPy and SciPy, complemented by C/C++ implementations. It is engineered to function seamlessly with other software and adapt to complicated workflows, including its application alongside modules like Qiskit Nature.

In the PySCF driver, you have the option to choose from a range of basis sets to define the atomic orbitals for your calculations. In most calculations, I use the STO-3G basis. It uses a linear combination of three Gaussian functions to approximate Slater-type orbitals(STO). This set is frequently referred to as minimal because it includes a solitary basis function for each orbital. It demonstrates considerable computational efficiency for simple system. For more complex ones, it provides very low accuracy.

Chapter 3

Computing ground-state energies of molecules

3.1 Hydrogen

The hydrogen molecule is used as a basic example to demonstrate the calculation of the ground-state energy with PySCF. Because of the structure of this molecule, which contains only two atoms and each one consists of one electron and one proton, the calculation time can be minimized, allowing the focus to remain on other aspects. Moreover, the analysis concentrates on how the results change depending on the choice of a variety of parameters.

After specifying molecular data (including geometry, unit, or basis set), invoking the `.run()` function causes the driver to return an `ElectronicStructureProblem`, which provides the fermionic operator as one of its attributes.

```
fermionic_op = ESP.hamiltonian.second_q_op()
```

Listing 3.1: Extraction of the fermionic operator from the class ElectronicStructureProblem

In the example results of the calculation of the operator 3.2, there are 4 different spin orbitals. A +_i operator adds and a -_i operator removes the fermion from state i. We can distinguish interactions on the same orbital (example: +_0 +_0 -_0 -_0) that often correspond to electron repulsion or different orbital (example: +_0 +_1 -_1 -_0) which can describe hopping between orbitals, forming a bond between atoms or more complex behaviors.

```
FermionicOp({'+_0 +_0 -_0 -_0': 0.33785507740175813, '+_0 +_1 -_1 -_0'
    : 0.33229086512764816, '+_0 +_2 -_2 -_0': 0.33785507740175813, '+
    _0 +_3 -_3 -_0': 0.33229086512764816, '+_0 +_0 -_1 -_1':
    0.0904655998921157, '+_0 +_1 -_0 -_1': 0.0904655998921157, '+_0 +
    _2 -_3 -_1': 0.0904655998921157, '+_0 +_3 -_2 -_1':
    0.0904655998921157, '+_1 +_0 -_1 -_0': 0.0904655998921157, '+_1 +
    _1 -_0 -_0': 0.0904655998921157, '+_1 +_2 -_3 -_0':
    0.0904655998921157, '+_1 +_3 -_2 -_0': 0.0904655998921157, '+_1 +
    _0 -_0 -_1': 0.33229086512764816, '+_1 +_1 -_1 -_1':
    0.34928686136600906, '+_1 +_2 -_2 -_1': 0.33229086512764816, '+_1
    +_3 -_3 -_1': 0.34928686136600906, '+_2 +_0 -_0 -_2':
    0.33785507740175813, '+_2 +_1 -_1 -_2': 0.33229086512764816, '+_2
    +_2 -_2 -_2': 0.33785507740175813, '+_2 +_3 -_3 -_2':
```

```
0.33229086512764816, '+_2 +_0 -_1 -_3': 0.0904655998921157, '+_2 +
_1 -_0 -_3': 0.0904655998921157, '+_2 +_2 -_3 -_3':
0.0904655998921157, '+_2 +_3 -_2 -_3': 0.0904655998921157, '+_3 +
_0 -_1 -_2': 0.0904655998921157, '+_3 +_1 -_0 -_2':
0.0904655998921157, '+_3 +_2 -_3 -_2': 0.0904655998921157, '+_3 +
_3 -_2 -_2': 0.0904655998921157, '+_3 +_0 -_0 -_3':
0.33229086512764816, '+_3 +_1 -_1 -_3': 0.34928686136600906, '+_3
+_2 -_2 -_3': 0.33229086512764816, '+_3 +_3 -_3 -_3':
0.34928686136600906, '+_0 -_0': -1.25633907300325, '+_1 -_1':
-0.47189600728114184, '+_2 -_2': -1.25633907300325, '+_3 -_3':
-0.47189600728114184}, num_spin_orbitals=4, )
```

Listing 3.2: Fermionic operator for the default driver settings

To convert this operator into the qubit Hamiltonian form (1.25) consisting of Pauli strings, it is necessary to employ a mapper. For parity mapper and default driver settings, which are represented by a hydrogen molecule with equilibrium bond length (0.735 Å), the Pauli terms and coefficients are demonstrated in (3.1). Jordan-Wigner mapping technique causes a triple increase in variables.

Table 3.1: Components of the `SparsePauliOp` for a two-qubit system of H_2

Term	**Coefficient**
II	−1.05237325
IZ	+0.39793742
ZI	−0.39793742
ZZ	−0.0112801
XX	+0.1809312

The largest contribution comes from the identity term II. Expressions IZ, ZI and ZZ correspond to interactions between the qubits, and XX is a coupling operator.

As noted previously, PySCF offers support for a wide range of basis sets in quantum chemistry computations. In addition to STO-3G, there are several other databases, and a comparison of the calculated values derived from these, as well as the associated computation times, is presented in 3.2. As computational precision improves, the processing time increases in the same way. However, for our purposes, which involve small molecules, the STO-3G basis set provides adequate accuracy.

Basis set	**Computation time**	**Calculated enery [Hatree]**
STO-3G	23.3 s	-1.1373060357533968
3-21G	54.8 s	-1.1477292427931434
6-31G	55.3 s	-1.1516143199365816
6-311G	7m 36 s	-1.153414664680148

Table 3.2: Comparison of basis sets by computation time and calculated value for the ground-state of hydrogen molecule with bond lenght 0.735 Å

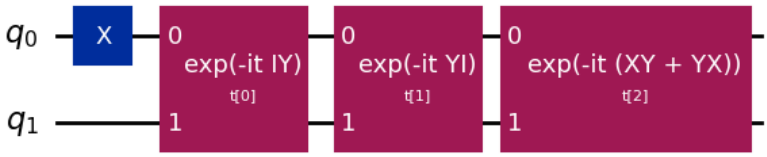
3.1.1 Circuits

There are a few types of circuit that can be constructed in Qiskit. The one widely employed in chemistry is a time-evolution circuit, which allows exploration of phase

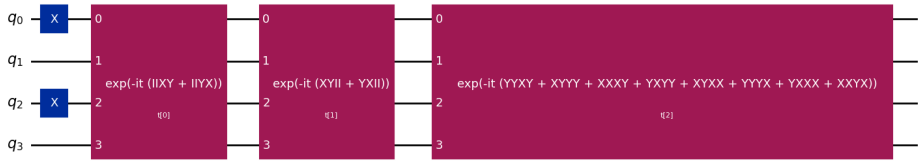
transitions. The parametrized circuit is particularly useful in VQE, where the circuit parameters are optimized to approximate the energy of the ground state. Circuits (3.2) are built using the Unitary Coupled Cluster with Single and Double Excitations (`UCCSD`) and the initial state of Hartree-Fock. They have 2 or 4 qubits, depending on the mapper selected. The system prepared using parity mapper (3.1(a)) has two wires that symbolize qubits (q_0, q_1). Rectangles represent operations performed on a particular qubit. The X-Pauli gate applied to the qubit q_0 prepares the Hartree-Fock state, and the remaining are unitary gates that represent:

1. exp(-it IY) - a single excitation,
2. exp(-it YI) - another single excitation,
3. exp(-it (XY+YX)) - double excitation, coupling the qubits.

Each of the three gates is parameterized by the t parameter that is optimized during the VQE process.



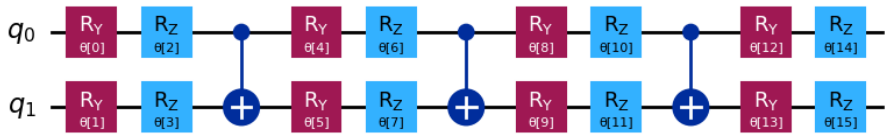
(a)



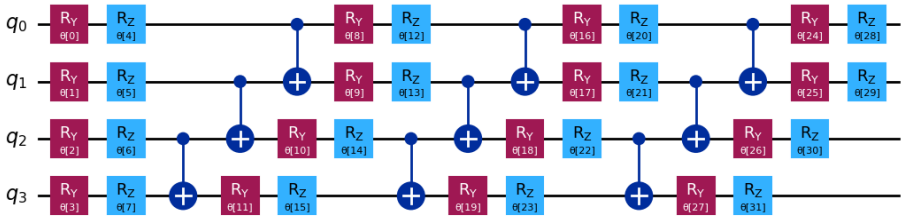
(b)

Figure 3.1: Time-evolution circuit UCCSD for hydrogen molecule using a) parity and b) J-W mapping

In the section dedicated to different simulators, I will also be using other types such as `EfficientSU2` ansatz (3.2(a)) which is also a parametrized circuit, but created using a specific pattern of rotations (single-qubit rotational gates R_Y and R_Z) and entangling gates (CNOT).



(a)



(b)

Figure 3.2: Diagrams of the EfficientSU2 circuit for a) 2 qubits and b) 4 qubits

3.1.2 Optimizers

VQE algorithm requires the classical optimizer to properly function. The performance of arbitrary selected three of them are compared in Table 3.3 and Figure 3.3. COBYLA (Constrained Optimization By Linear Approximation) optimizer needs at least two times more number of iterations than others. Taking accuracy into account, the L-BFGS-B (Limited-memory BFGS Bound optimizer) and SLSQP (Sequential Least SQuares Programming) optimizers provide results that are more coherent with the theoretical approximate value. They also exhibit a smoother convergence, in contrast to COBYLA, which shows significant energy oscillations.

Table 3.3: Comparison of performances of different optimizers in computing electronic energy on simulator for H_2 molecule with bond lenght 0.735 Å

Name	**Type**	**Iteration**	**Energy[Hartree]**
opt1	L_BFGS_B	20	-1.8572750302023735
opt2	COBYLA	44	-1.8572749970474753
opt3	SLSQP	10	-1.8572750301445968

The L-BFGS-B or the SLSQP optimizers are employed for the majority of calculations, as they are well-suited for this class of problems because of the balance between accuracy and the number of iterations needed to converge.

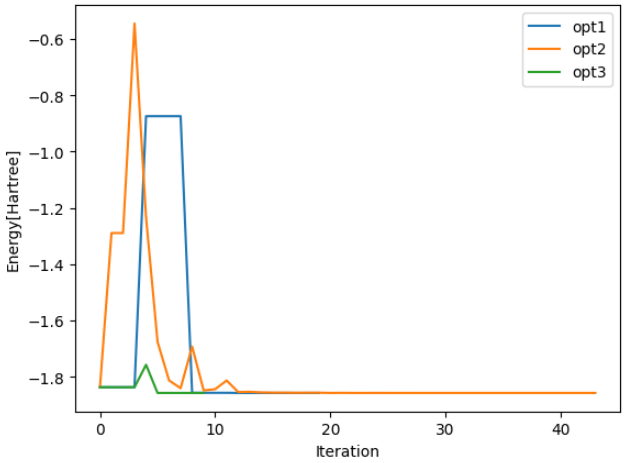
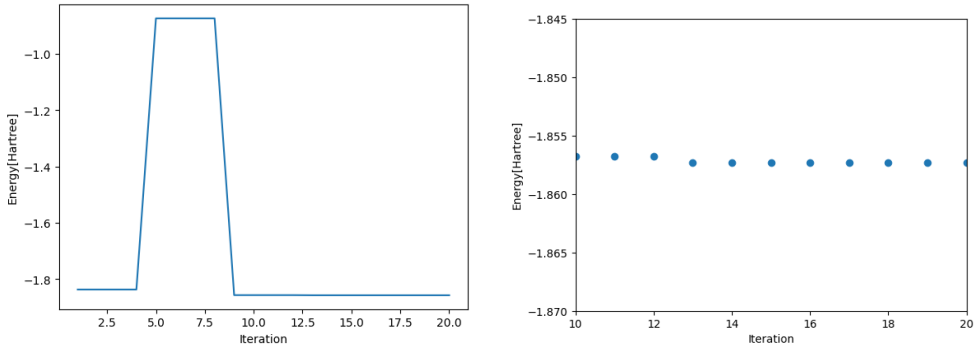


Figure 3.3: Behavior of three different classical optimizers applied in the VQE algorithm, detailed information in table 3.3



(a) (b)

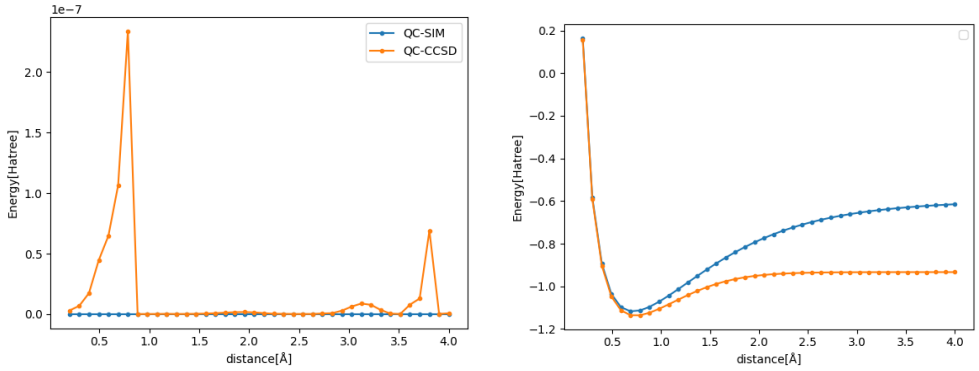
Figure 3.4: a) The energy convergence for the L-BFGS-B optimizer during VQE calculations of H_2(bond lenght = 0,735 Å) on the ibm_kyiv backend, b) a zoomed-in view of the last 10 iterations

3.1.3 Results

The energy derived from VQE for a hydrogen molecule with a bond length of 0.735 was calculated using a simulator and a quantum computer. Furthermore, the ground state was determined numerically with the self-consistent field approach. To enhance the precision of this result, the CCSD method was applied. Comparison 3.4 shows that the simulator and Kyiv's quantum backend returned identical results, demonstrating their coherence in calculating the energy of the ground-state of the hydrogen molecule. The numerical result only approaches two of the latter results after the improvement of accuracy.
The energy results for other bond lengths were also analyzed (3.5(a)) by subtracting the values obtained numerically and in the simulator from those obtained in the quantum computer. The difference between the outcomes of the simulator and the

Table 3.4: Comparison of different calculations of the ground-state energy for the H_2 molecule with bond length 0.735 Å

Energy[Hartree]	**Calculation**
-1.137306035753395	simulator
-1.137306035753395	ibm_kyiv
-1.116998996754	SCF
-1.137306193391968	CCSD



(a) (b)

Figure 3.5: Presentation of the discrepancy in results between different methods or devices: subtraction of quantum computer (QC) and a simulator's (SIM) or numerical results (CCSD), b) numerical method with and without error correction

quantum hardware is negligible. When it comes to the numerically obtained results, they are very close to those of the quantum backend in the range of 1 to 3 Å. Beyond this range, differences of up to seven decimal places appear. In Figure 3.5(b), we can see how the previously obtained numerical results were corrected using USSD. The distance of 0.735 is a theoretical equilibrium bond length for the hydrogen molecule. This value can be found by checking its length at the lowest energy. At the beginning, energies for several points were calculated. The range in which the point with the lowest energy is located was selected (0.72-0.76 Å) and then the energies were computed again for this interval (3.6). The minimum value was reached for a length of 0.734949 Å, which is approximately equal to the theoretical result.

Figure 3.7 illustrates the energy components and the total energy as a function of distance. The absolute value of electronic energy for lengths smaller than 1 Å is the highest (approximately 2 Hatree $\approx$ 54 eV). Nuclear repulsion energy also decreases at the beginning, but more rapidly. This happens because of the strong electrostatic repulsion of positively charged nuclei, which decreases monotonically as the nuclei are farther apart. Then both components gradely stabilize. In conclusion, with an increase of bond length, the total energy approaches a more stable value. When distance reaches the dissociation limit, atoms are too far to interact and the molecule breaks. The strongest bond in a hydrogen molecule occurs at equilibrium

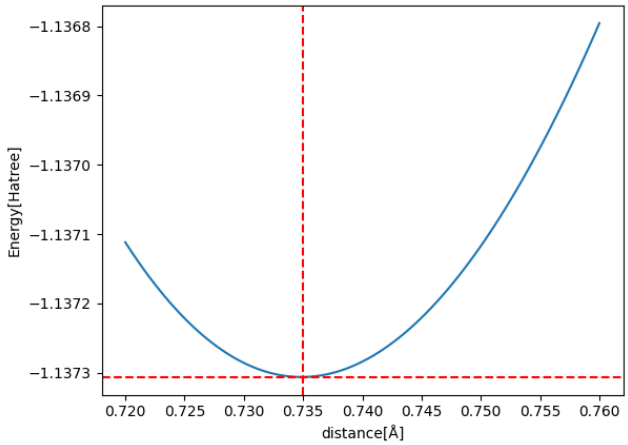


Figure 3.6: The process of finding the equilibrium bond length for hydrohen molecule

bond length.

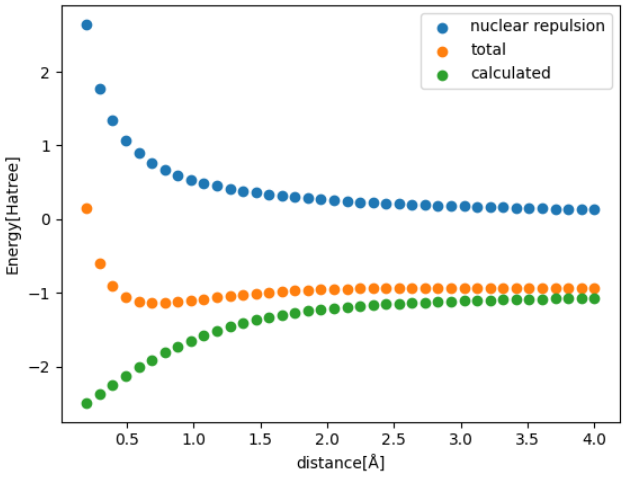
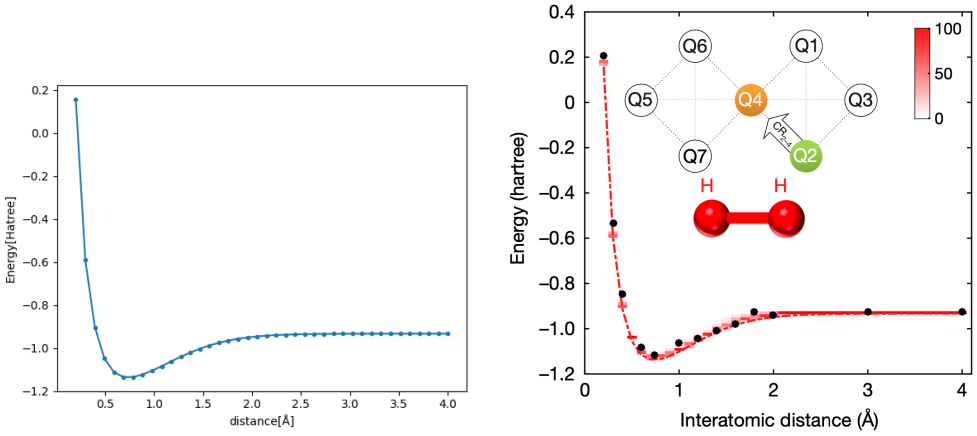


Figure 3.7: Plot of energy components and total ground-state energy as a function of interatomic distance for H_2 calculated on Kyiv backend

To verify the accuracy of our computation, we can compare our results with those presented in [28].



(a) (b)

Figure 3.8: Calculated ground-state energy of hydrogen molecule for variety of interatomic distances a) achived by quantum backend kyiv compared with b) the results published in the experimental Nature Letter article [28] showing very good agreement.

3.1.4 Simulators

The previously mentioned simulator is the default simulator provided by the Qiskit Estimator class. Furthermore, there exist additional simulators that differ in both precision and efficiency. The fake provider module [29] offers classes designed to replicate the behavior of IBM's backends by utilizing system snapshots. These snapshots include key details about the quantum system. Aersimulators [30] are much more recent tools, yielding outcomes that are closer to those obtained from quantum systems (3.5) at reduced number of iterations.

Table 3.5: Comparison of calculations on simulators of the minimal energy of 2-qubit system

Energy[Hartree]	**Simulator type**
-1.1370877207753045	Aer
-1.1174118721616226	Fake provider

This behavior is demonstrated one the Figure 3.9 that were obtained using the ansatz 3.2(a) and the implementation of the VQE algorithm from [31]. The employed FakeKyiv backend mocks the properties of Kyiv quantum processor. The automatic mode aersimulator identifies the most optimal technique by analyzing the characteristic of the quantum circuit under execution.

Other computations were conducted on fake provider module backends with varying numbers of qubits. It turned out that higher-quality back-ends (FakeKyiv) have better stability and accuracy of the results.

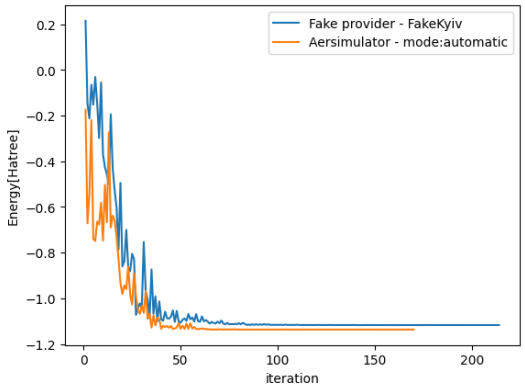


Figure 3.9: Convergence of the ground energy of the hydrogen molecule calculated using VQE algorithm for two types of simulators: Aersimulator in automatic mode and FakeKyiv from fake provider module

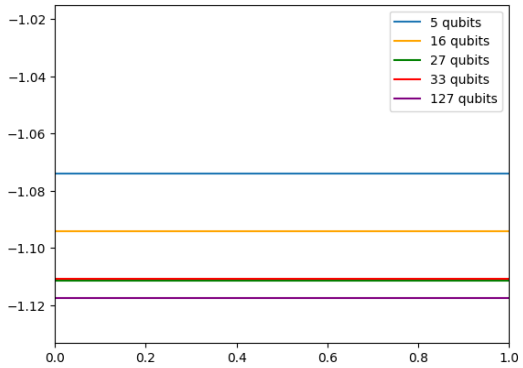
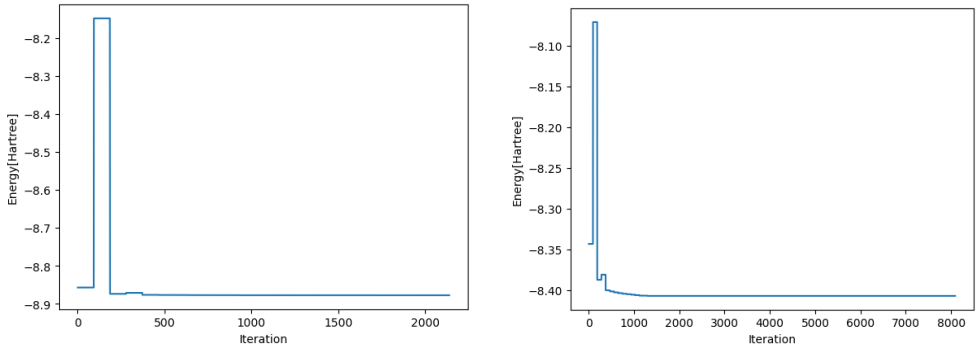


Figure 3.10: Ground state energies obtained from simulated fake-backends mimicking existing quantum processors employing different number of qubits: 5 - FakeAthensV2, 16 - FakeGuadalupeV2, 27 - FakeAuckland, 33 - FakePrague, 127 - FakeKyiv.

3.2 Analysis in relation to other molecules

Lithium hydride(LiH) and dilithium (Li_2) molecules are widely used in quantum chemistry as benchmarks, including the Monte Carlo computational technique [32], which examines the use of quantum annealers [33] and also for the VQE algorithm but for different purposes, such as computing the Stark effect [34]. LiH is the smallest molecule containing two nonidentical atoms, but Li_2 despite being homogeneous also captures the interest of researchers.
There is a significant growth in the complexity of an electronic structure problem for these molecules. We can observe it in the increase in the number of iterations that goes from 20 for H_2 (3.4(a)) to more than 2000 for LiH (3.11(a)) and even more than 8000 for Li_2. The time required to perform calculations using the VQE technique also increases from a minute to an hour and eventually to several hours and days.

Table 3.6 provides a comparison between the number of Pauli terms and qubits



(a) (b)

Figure 3.11: Convergence of energies for a)LiH b)Li_2 in their equilibrium bond lengths a)1.595 Å, b) 2.673 Å calculated on the default simulator (result for Li_2 were performed with optimizer with low tolerance)

required using different mappings. The parity mapper reduced the number of qubits by two in each case. The number of Pauli terms remains the same for both mappers in larger molecules(LiH and Li_2), but there is a significant reduction for H_2 from 15 to 5. As the size of the molecules increases, the number of Pauli terms shows exponential growth 3.12.

Table 3.6: Comparison of Pauli terms and qubits required for different molecules using J-W Mapper and Parity Mapper

Molecule	**J-W mapper**		**Parity mapper**	
	Pauli Terms	Qubits	Pauli Terms	Qubits
H_2	15	4	5	2
LiH	631	12	631	10
Li_2	2951	20	2951	18

For the LiH molecule, the dependence of time and the number of iterations on the interatomic distance was also examined (3.13). These quantities exhibit analogous patterns as they are directly related to each other. Interestingly, values close to the equilibrium bond length show a significant decrease in the number of iterations, dropping by an order of magnitude compared to the highest values, while the computation time is reduced by up to five times. The closer the initial guess is to the equilibrium, the less the algorithm needs to adjust. Consequently, the algorithm converges faster, requiring fewer iterations.

The illustrations 3.14 depict the energy change associated with the bond lengths of lithium hydride. In the graph 3.14(b) black points represent experimental results, while a dashed green line denotes the exact energy curve and a heatmap shows the outcomes of numerical simulations (plot taken from obtained in the experimental Nature Letter article [28] for comparison reasons). Figure 3.14(a) provides the data calculated within the thesis using the most precise simulator. In both examples, the energy decreases significantly as the bond length approaches approximately 1.6

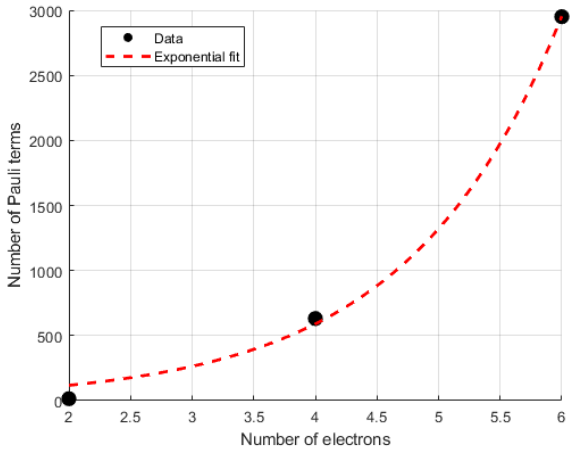
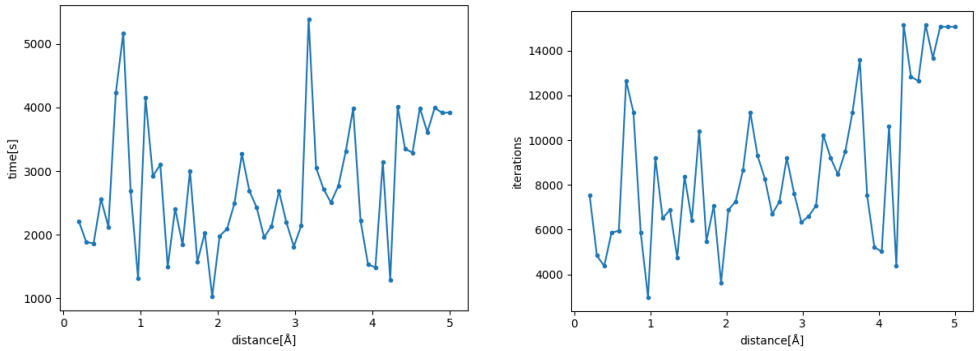


Figure 3.12: The plot illustrating number of electrons in the system and corresponding number of Pauli terms, along with the corresponding exponential function fitted to data

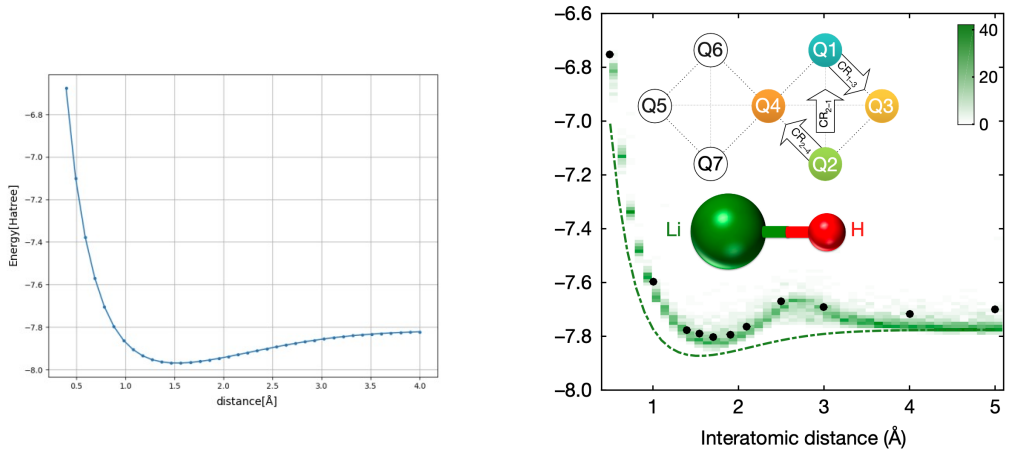


(a) (b)

Figure 3.13: Graph illustrating the a) time b) iterations needed for energy convergence plotted against the interatomic distance of LiH molecule

Å. Beyond this distance, the energy gradually increases until it attains a stable value. The equilibrium bond length for LiH is more than twice that in H_2, due to the presence of a larger Li atom. LiH curve is wider, indicating a weaker bond compared to the H_2 one 3.8, which is sharp and narrow and represents a strong and stiff bond.

To compare the changes in energy, numerical calculations (3.7) were performed for Li_2 and LiH. The difference between the ground-state energy for both systems is significant, as the ground-state energy for Li_2 is almost two times smaller. For LiH, the computed values for the simulator were presented, which are shown in the graph 3.11(a). The simulator results for Li_2 3.11(b) are not considered as they were obtained just to demonstrate the scaling of the problem when using an optimizer with low tolerance, specifically `L_BFGS_B(ftol=1, maxfun=2, maxiter=2)`.



(a) (b)

Figure 3.14: The calculation of the ground-state energy for LiH across various bond lengths that was a) achieved using the default simulator, and b) documented in the experimental Nature Letter article [28]

Table 3.7: Energy components (in Hartree) calculations for LiH, Li_2 with different methods

Molecule	**Method**	**Electronic energy (ground state)**	**Nuclear repulsion**	**Total energy**
LiH	SCF	-8.857341498221158	0.995317638094044	-7.86202386012711
LiH	CCSD	-8.877709075710472	0.995317638094044	-7.882391437616428
LiH	the default simulator	-8.87770892902312	0.995317638094044	-7.882391290929076
Li_2	SCF	-16.42046663753356	1.7817414509090912	-14.6387251866245
Li_2	CCSD	-16.4490014602581	1.7817414509090912	-14.6672600093

Chapter 4

Replacement for drivers

As mentioned above, reliance on external libraries requires their installation, which can be time-consuming and may introduce potential complexities. For this reason, we can create an independent driver that serves as a substitute for these libraries. For now, the calculations are based on tabulated electronic integrals. They are calculated in advance and stored, ensuring immediate access to the values.

The driver is based on `ElectronicStructureDriver`, so it must have the same structure. It requires three requested abstract methods and a constructor of a class to work properly.

1. `to_qschema` - converts molecular and computational data into a `QCSchema` format.
2. `to_problem` changes output of `to_qschema` into `ElectronicStructureProblem`.
3. `run` delegates work to another method - it calls `self.to_problem()`.
4. `__init__` is responsible for setting up a functional instances of a class (for example, chosen bond length of molecule).

In the `to_qschema`, appropriate values are assigned to the data container `data = _QCSchemaData()`. The number of parameters has been minimized to the essential one to calculate the ground-state energy. The chosen properties represent:

- `hij_mo` and `eri_mo` - one-electron and two-electron integrals in the molecular orbital basis,
- `nmo` - total number of molecular orbitals,
- `nalpha` and `nbeta` - number of spin-up and number of spin-down electrons,
- `coords` - coordinates od the nuclei,
- `symbols` - chemical symbols of the atoms in the molecule,
- `e_nuc` - nuclear repulsion energy.

In this study, the construction of a driver is investigated using the hydrogen molecule. The one-electron and two-electron integrals are computed iteratively by PySCF for the specified range of bond lengths. They are saved to a file, allowing them to be loaded later in the custom library developed as a part of this thesis and future work.

```
def to_qcschema(self) -> QCSchema:

        #Creating data container
        data_H2 = _QCSchemaData()

        current = os.path.dirname(__file__)
        path_len = os.path.join(current, "lenghts_list.npy")

        #Loading array with bond lenghts
        loaded_list = np.load(path_len, allow_pickle=True)

        #Finding the index of the nearest tabulated bond length
        #to the value entered to the driver
        R = self._bond_lenght
        i = (np.abs(loaded_list - R)).argmin()

        #Loading arrays with integral coefficients
        path_h1 = os.path.join(current, "H2/h1_a_list.npy")
        path_h2 = os.path.join(current, "H2/h2_aa_list.npy")
        loaded_matrix_list1 = np.load(path_h1,
                                      allow_pickle=True,
                                      mmap_mode="r")
        loaded_matrix_list2 = np.load(path_h2,
                                      allow_pickle=True,
                                      mmap_mode="r")

        #Selecting the nearest results
        data_H2.hij_mo= loaded_matrix_list1[i]
        data_H2.eri_mo= loaded_matrix_list2[i]
```

Listing 4.1: Fragment of to_qcsschema that handles data loading and selects the appropriate values

The expression for the nuclear repulsion energy from 1.3 for H_2 simplify to:

$$U_{nn} = \frac{1}{R}. \tag{4.1}$$

R is a bond length that must be provided during driver initialization. The given value needs to be converted from a Angstrom to a Bohr unit.

```
        bohr = 0.529177249 # 1 bohr ->  0.529177249 Angstrom
        R = R/bohr
        data_H2.e_nuc=1/R
```

Listing 4.2: Unit conversion and calculating nuclear repulsion energy

Then we set up the remaining variables.

```
        data_H2.symbols=['H', 'H']
        data_H2.coords=[0.0, 0.0, 0.0, 0.0, 0.0, R]
        data_H2.nmo=2
        data_H2.nalpha=1
        data_H2.nbeta=1
```

Listing 4.3: Initializing the rest of the variables

Once all elements are created, we can use this driver as a replacement. To make things easier, it is best to limit all input values so that complex strings such as `atom` in 2.2 do not appear.

```
import fakedriver
driver = fakedriver.H2_driver(bond_lenght= 0.735)
ESP = driver.run()
```

Listing 4.4: Example usage of created library

The results it produces are directly tied to the amount of pre-tabulated data available. Expanding the amount of data could improve its utility, but it also increases storage requirements. The recommended approach is to select a range of bond lengths that covers the expected physical configurations of the molecule. For the hydrogen molecule, the equilibrium length is approximately 0.735 Å, while beyond 4 Å system approaches the dissociation limit. It is advisable to use smaller steps closer to the equilibrium region, because of rapid changes of the electronic structure. The distances at which the interactions between atoms become negligible are not worth keeping. All initial goals have been met and the initial version of self-sufficient library which can be a part of the Qiskit framework.

In the future, I would like to focus on replacing data with real-time approximation. It will reduce memory requirements and allow for greater flexibility and adaptability. However, this requires a deep understanding of the estimations performed by PySCF. My goal will not be to obtain highly accurate results, but rather results that are sufficiently good to maintain similar proportions and follow similar trends relative to each other. Specifically, I will try to explore the complexity of various problems and their impact on efficiency and scalability. Another extension would be to add the ability to consider application of molecules with a larger number of atoms and varying geometries.

Summary

Quantum computing is undoubtedly an extremely challenging yet fascinating topic to explore and elaborate upon. This thesis examines merely a segment of this extensive domain. Initially, it addresses the history and theoretical foundations of quantum computers and quantum chemistry. Following this, the document illustrates the specific functionalities available through the utilization of quantum software packages such as Qiskit, Qiskit Nature, and PySCF. The analysis for fermionic and qubit representations of the Hamiltonian for the electronic structure problem is presented. The impact of various mapping techniques and optimization strategies on the results obtained is also discussed. Additionally, a range of different fake backends offered by Qiskit is showcased. Beyond the hydrogen molecule, which was considered in all calculations, the research was added for two larger molecules LiH and Li_2 for comparison. Finally, the structure of designing a custom self-sufficient driver based on the `ElectronicStructureDriver` is explored and succesfully tested, along with discussing a few avenues for its future improvements and enhancements.

The outcomes of this research, along with any forthcoming advancements, will be stored and accessible through the repository on GitHub. The repository will be located at `https://github.com/m-martyna/thesis_qiskit_nature`. The library will be made public at a subsequent time. It will also provide a link to the fork of Qiskit Nature related to `ElectronicStructureDriver`.

Bibliography

[1] Richard P. Feynman. "Simulating Physics with Computers". In: *International Journal of Theoretical Physic* (1982). DOI: https://doi.org/10.1007/BF02650179.

[2] A. M. Turing. "On Computable Numbers, with an Application to the Entscheidungsproblem". In: *Proceedings of the London Mathematical Society* s2-42.1 (1937), pp. 230–265. DOI: https://doi.org/10.1112/plms/s2-42.1.230. eprint: https://londmathsoc.onlinelibrary.wiley.com/doi/pdf/10.1112/plms/s2-42.1.230. URL: https://londmathsoc.onlinelibrary.wiley.com/doi/abs/10.1112/plms/s2-42.1.230.

[3] David Deutsch and Roger Penrose. "Quantum theory, the Church–Turing principle and the universal quantum computer". In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 400.1818 (1985), pp. 97–117. DOI: 10.1098/rspa.1985.0070. eprint: https://royalsocietypublishing.org/doi/pdf/10.1098/rspa.1985.0070. URL: https://royalsocietypublishing.org/doi/abs/10.1098/rspa.1985.0070.

[4] Luo-Kan Chen et al. "Observation of ten-photon entanglement using thin BiB3O6 crystals". In: *Optica* 4.1 (Jan. 2017), pp. 77–83. DOI: 10.1364/OPTICA.4.000077. URL: https://opg.optica.org/optica/abstract.cfm?URI=optica-4-1-77.

[5] C. J. Ballance et al. "High-Fidelity Quantum Logic Gates Using Trapped-Ion Hyperfine Qubits". In: *Physical Review Letters* 117.6 (Aug. 2016). ISSN: 1079-7114. DOI: 10.1103/physrevlett.117.060504. URL: http://dx.doi.org/10.1103/PhysRevLett.117.060504.

[6] Ryan Babbush et al. "Low-Depth Quantum Simulation of Materials". In: *Phys. Rev. X* 8 (1 Mar. 2018), p. 011044. DOI: 10.1103/PhysRevX.8.011044. URL: https://link.aps.org/doi/10.1103/PhysRevX.8.011044.

[7] Youngseok Kim et al. "Evidence for the utility of quantum computing before fault tolerance". In: *Nature* 618 (June 2023), pp. 500–505. DOI: 10.1038/s41586-023-06096-3.

[8] Sam McArdle et al. "Quantum computational chemistry". In: *Reviews of Modern Physics* 92.1 (Mar. 2020). ISSN: 1539-0756. DOI: 10.1103/revmodphys.92.015003. URL: http://dx.doi.org/10.1103/RevModPhys.92.015003.

[9] V.M. Ramaglia. "Electronic Structure (Theory): Atoms". In: *Encyclopedia of Condensed Matter Physics*. Ed. by Franco Bassani, Gerald L. Liedl, and Peter Wyder. Oxford: Elsevier, 2005, pp. 67–75. ISBN: 978-0-12-369401-0. DOI: https://doi.org/10.1016/B0-12-369401-9/00440-X. URL: https://www.sciencedirect.com/science/article/pii/B012369401900440X.

[10] Yudong Cao et al. "Quantum Chemistry in the Age of Quantum Computing". In: *Chemical Reviews* 119.19 (Aug. 2019), pp. 10856–10915. ISSN: 1520-6890. DOI: 10.1021/acs.chemrev.8b00803. URL: http://dx.doi.org/10.1021/acs.chemrev.8b00803.

[11] Gerald Mahan. "Many-Particle Physics". In: Jan. 2000. ISBN: 978-1-4419-3339-3. DOI: 10.1007/978-1-4757-5714-9_3.

[12] Alexander Altland and Ben D. Simons. *Condensed Matter Field Theory*. 2nd ed. Cambridge University Press, 2010.

[13] Jacob T. Seeley, Martin J. Richard, and Peter J. Love. "The Bravyi-Kitaev transformation for quantum computation of electronic structure". In: *The Journal of Chemical Physics* 137.22 (Dec. 2012). ISSN: 1089-7690. DOI: 10.1063/1.4768229. URL: http://dx.doi.org/10.1063/1.4768229.

[14] Andrew Tranter et al. "A Comparison of the Bravyi–Kitaev and Jordan–Wigner Transformations for the Quantum Simulation of Quantum Chemistry". In: *Journal of Chemical Theory and Computation* 14.11 (2018), pp. 5617–5630. ISSN: 1549-9618. DOI: 10.1021/acs.jctc.8b00450. URL: https://doi.org/10.1021/acs.jctc.8b00450.

[15] Brent Harrison et al. *Reducing the qubit requirement of Jordan-Wigner encodings of N-mode, K-fermion systems from N to* $\lceil \log_2 \binom{N}{K} \rceil$. 2023. arXiv: 2211.04501 [quant-ph]. URL: https://arxiv.org/abs/2211.04501.

[16] Pranav Gokhale et al. *Minimizing State Preparations in Variational Quantum Eigensolver by Partitioning into Commuting Families*. 2019. arXiv: 1907.13623 [quant-ph]. URL: https://arxiv.org/abs/1907.13623.

[17] Alberto Peruzzo et al. "A variational eigenvalue solver on a photonic quantum processor". In: *Nature Communications* 5.1 (July 2014). ISSN: 2041-1723. DOI: 10.1038/ncomms5213. URL: http://dx.doi.org/10.1038/ncomms5213.

[18] Ilya G. Ryabinkin et al. *Qubit coupled-cluster method: A systematic approach to quantum chemistry on a quantum computer*. 2018. arXiv: 1809.03827 [quant-ph]. URL: https://arxiv.org/abs/1809.03827.

[19] Jonathan Romero et al. *Strategies for quantum computing molecular energies using the unitary coupled cluster ansatz*. 2018. arXiv: 1701.02691 [quant-ph]. URL: https://arxiv.org/abs/1701.02691.

[20] Vladyslav Verteletskyi, Tzu-Ching Yen, and Artur F. Izmaylov. "Measurement optimization in the variational quantum eigensolver using a minimum clique cover". In: *The Journal of Chemical Physics* 152.12 (Mar. 2020). ISSN: 1089-7690. DOI: 10.1063/1.5141458. URL: http://dx.doi.org/10.1063/1.5141458.

[21] Tzu-Ching Yen, Vladyslav Verteletskyi, and Artur F. Izmaylov. *Measuring all compatible operators in one series of a single-qubit measurements using unitary transformations*. 2020. arXiv: 1907.09386 [quant-ph]. URL: https://arxiv.org/abs/1907.09386.

[22] *Qiskit*. URL: https://github.com/Qiskit/qiskit/tree/main. (accessed: 2024).

[23] Jay M. Gambetta, Jerry M. Chow, and Matthias Steffen. "Building logical qubits in a superconducting quantum computing system". In: *npj Quantum Information* 3.1 (Jan. 2017). ISSN: 2056-6387. DOI: 10.1038/s41534-016-0004-0. URL: http://dx.doi.org/10.1038/s41534-016-0004-0.

[24] J. A. Schreier et al. "Suppressing charge noise decoherence in superconducting charge qubits". In: *Physical Review B* 77.18 (May 2008). ISSN: 1550-235X. DOI: 10.1103/physrevb.77.180502. URL: http://dx.doi.org/10.1103/PhysRevB.77.180502.

[25] *Qiskit Nature*. URL: https://doi.org/10.5281/zenodo.7828767. (accessed: 2024).

[26] *PySCF*. URL: https://pyscf.org/index.html. (accessed: 2024).

[27] Qiming Sun et al. "The Python-based Simulations of Chemistry Framework (PySCF)". In: *Wiley Interdisciplinary Reviews: Computational Molecular Science* 8 (Jan. 2017). DOI: 10.1002/wcms.1340.

[28] Abhinav Kandala et al. "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets". In: *Nature* 549.7671 (Sept. 2017), pp. 242–246. ISSN: 1476-4687. DOI: 10.1038/nature23879. URL: http://dx.doi.org/10.1038/nature23879.

[29] *Fake Provider*. URL: https://docs.quantum.ibm.com/api/qiskit-ibm-runtime/fake_provider. (accessed: 2024).

[30] *Aersimulator*. URL: https://qiskit.github.io/qiskit-aer/. (accessed: 2024).

[31] *Variational quantum eigensolver*. URL: https://learning.quantum.ibm.com/tutorial/variational-quantum-eigensolverl. (accessed: 2024).

[32] Saeed Nasiri and Mansour Zahedi. "A benchmark study of Li2+, Li2, LiH+ and LiH: Quantum Monte-Carlo and coupled-cluster computations". In: *Computational and Theoretical Chemistry* 1114 (2017), pp. 106–117. ISSN: 2210-271X. DOI: https://doi.org/10.1016/j.comptc.2017.05.015. URL: https://www.sciencedirect.com/science/article/pii/S2210271X17302426.

[33] Scott N. Genin, Ilya G. Ryabinkin, and Artur F. Izmaylov. *Quantum chemistry on quantum annealers*. 2019. arXiv: 1901.04715 [physics.chem-ph]. URL: https://arxiv.org/abs/1901.04715.

[34] Carlos Tavares et al. "Quantum simulation of the ground-state Stark effect in small molecules: a case study using IBM Q". In: *Soft Computing* 25.9 (Jan. 2021), pp. 6807–6830. ISSN: 1433-7479. DOI: 10.1007/s00500-020-05492-5. URL: http://dx.doi.org/10.1007/s00500-020-05492-5.